IF3070 DASAR INTELEGENSI ARTIFISIAL
TUGAS BESAR 1



Local Search untuk Penempatan Kendaraan pada Geladak Kapal Kargo

Kelompok 17: NEXt

Agung Ahmad Rifai	18224042
Muhammad Zulfa Fauzan Nurhuda	18224064
Bram Sebastian Pangaribuan	18224076

Program Studi Sistem dan Teknologi Informasi
Sekolah Teknik Elektro dan Informatika Institut Teknologi Bandung
Oktober 2026

Daftar Isi

Daftar Isi **2**

BAB I
DESKRIPSI PERSOALAN **4**

1. Latar Belakang 4
2. Definisi Masalah 4
3. Definisi Masalah 5
4. Cakupan Perkerjaan 7

BAB II
PEMBAHASAN **8**

1. Representasi Masalah dan Fungsi Pendukung 8
1.1. Representasi State dan Move 8
1.2. Prinsip Feasible-Only dan Pengecekan Constraint 11
1.3. Fungsi Objektif 11
1.4. State Awal Acak 12
1.5. Fungsi Successor 13
2. Hill-Climbing 16
1.1. Steepest Ascent Hill-Climbing 19
1.2. Stochastic Hill-Climbing 21
1.3. Hill-Climbing dengan Sideways Move 22
1.4. Random Restart Hill-Climbing 25
3. Simmulated Annealing 29
4. Genetic Algorithm 33

BAB 4
ANALISIS DAN HASIL EKSPERIMEN **45**

1. Ringkasan Hasil 45
2. Seberapa Dekat dengan Optimum Global? 46
3. Perbandingan Hasil dan Konsistensi 47
4. Perbandingan Durasi 48
5. Pengaruh Banyak Iterasi dan Jumlah Populasi pada GA 49
6. Keterbatasan 50

BAB 5
KESIMPULAN DAN SARAN **51**

BAB 6
PEMBAGIAN TUGAS **52**

BAB 7

REFERENSI..... 53
BAB 8
LAPORAN FORM PENGGUNAAN GENERATIVE AI..... 54

BAB I
DESKRIPSI PERSOALAN

1. Latar Belakang

Sebuah perusahaan pelayaran mengoperasikan kapal kargo pengangkut kendaraan. Pada setiap keberangkatan, perusahaan menerima daftar kendaraan dengan dimensi, berat, tarif pengiriman (`shipping_fee`) dan perkiraan waktu tiba (`eta`) yang berbeda-beda. Penempatan kendaraan pada geladak selama ini dilakukan manual sehingga ruang geladak tidak termanfaatkan dengan baik. Tugas ini membangun sistem pendukung keputusan yang **menentukan kendaraan mana yang diangkut** serta **posisi dan orientasinya** pada geladak, dengan algoritma *local search*.

2. Definisi Masalah

Geladak kapal dan setiap kendaraan didefinisikan sebagai grid dua dimensi berbentuk persegi atau persegi panjang.

Entitas	Atribut	Keterangan
`vehicle`	`id, width, length`	Tanda pengenal unik dan ukuran (satuan sel grid)
	`shipping_fee`	Tarif atau keuntungan pengangkutan (juta rupiah)
	`weight`	Berat (satuan unit)
	`eta`	Sisa hari sampai ke tujuan.
	`orientasi`	*Horizontal* atau *Vertical*.
`ship`	`width, length`	Ukuran geladak (sel grid).
	`max_capacity`	Batas total berat yang boleh diangkut.

Fungsi objektif: Memaksimalkan total `shipping_fee` kendaraan yang diangkut.

Batasan (*constraint*):

1. Tidak ada dua kendaraan menempati sel yang sama (*no overlapping*).
2. Seluruh sel setiap kendaraan yang diangkut berada di dalam geladak.
3. Total berat kendaraan yang diangkut tidak melebihi `max_capacity`.

Kendaraan boleh tidak diangkut (seluruhnya di luar kapal) dan *state* awal dibuat acak. *Move* yang diizinkan pada setiap iterasi adalah:

1. Menukar dua kendaraan (di dalam maupun di luar kapal).
2. Memindahkan sebuah kendaraan ke koordinat yang berbeda (termasuk mengeluarkan atau memasukkannya).
3. Mengubah orientasi sebuah kendaraan.

3. Definisi Masalah

Instance yang digunakan adalah *instance default* (src/instances/default.json) dengan prinsip batas luas dan batas berat sama-sama mengikat. Sehingga, tidak ada satu batasan yang membuat batasan lain tidak relevan.

Besaran	Nilai
Ukuran geladak	10 × 20 = 200 sel
Kapasitas berat	72
Jumlah kendaraan	25 (Car01 sampai Car25)
Ukuran kendaraan	1 - 4 × 2 - 6
Total luas kendaraan	300 sel (1,5 kali luas geladak)
Total berat	120 (kapasitas = 60% dari total)
Total `shipping_fee` semua kendaraan	1.462
Rentang `shipping_fee`, `weight`, `eta`	20 - 100, 2 - 8, 1 - 10

id	*(width)x(length)*	*fee*	berat	*eta*
Car01	2x6	60	5	3
Car02	3x4	75	6	50
Car03	4x3	45	4	2
Car04	2x6	80	7	7
Car05	3x4	35	3	1
Car06	4x3	55	5	4
Car07	4x6	25	2	10
Car08	2x6	95	8	6
Car09	4x6	70	6	3
Car10	3x6	40	4	8
Car11	2×5	65	5	5
Car12	3×5	30	3	9
Car13	4×5	90	7	2
Car14	2×4	50	4	6
Car15	3×3	85	6	4
Car16	1×4	20	2	10
Car17	4×4	100	8	1
Car18	2×3	58	5	7
Car19	3×2	42	3	8
Car20	1×2	33	4	5

id	*(width)x(length)*	*fee*	berat	*eta*
Car21	4×2	77	6	3
Car22	2×6	68	7	9
Car23	3×4	28	2	6
Car24	4×3	88	5	2
Car25	2×5	48	3	4

4. Cakupan Perkerjaan

Komponen	Status
Hill-climbing: *steepest-ascent*, *sideways-move*, *stochastic*, *random-restart*	Dikerjakan (termasuk bonus 2)
Simulated annealing	Dikerjakan
Genetic algorithm (3 variasi iterasi dan 3 variasi populasi)	Dikerjakan
Objective alternatif urgency-weighted-fee	Dikerjakan (bonus 1)
Multi-kapal	Tidak dikerjakan
Visualisasi	Dikerjakan

BAB II
PEMBAHASAN

Bab ini menjelaskan representasi dan fungsi pendukung yang dipakai di semua algoritma (bagian 1), implementasi tiap algoritma beserta percobaannya (bagian 2 sampai bagian 4), visualisasi program (bagian 5), analisis lintas algoritma (bagian 6), dan bonus yang diimplementasikan (bagian 7).

1. Representasi Masalah dan Fungsi Pendukung

1.1. Representasi *State* dan *Move*

Sebuah ***state*** adalah pemetaan setiap kendaraan ke letaknya. `Placement` berisi koordinat sel kiri atas `(x, y)` (bilangan bulat) dan orientasi, nilainya `None` jika kendaraan berada di luar kapal. *State* bersifat *immutable*, setiap perubahan menghasilkan *state* baru.

Konvensi orientasi:

- `Vertical`: lebar kendaraan searah sumbu-X dan panjangnya searah sumbu-Y
- `Horizontal`: kebalikannya.

Sebuah ***move*** adalah tuple `Change(vehicle_id, before, after)`. memutar dan memindahkan berisi satu `Change`, menukar berisi dua `Change` (karena *affect* ke dua kendaraan). `before` dan `after` adalah letak sebelum dan sesudah (`None` berarti di luar kapal). Representasi yang sama dipakai sebagai isi langkah pada berkas hasil sehingga viewer dapat memutar ulang pencarian.

Final state pada berkas hasil memuat daftar kendaraan di dalam kapal beserta letaknya, daftar kendaraan di luar kapal, nilai objektif, dan total `shipping_fee`.

```python
# src/core/models.py
type VehicleId = str


class Orientation(StrEnum):
    HORIZONTAL = "Horizontal"
    VERTICAL = "Vertical"
```

```python
ORIENTATIONS: Final = tuple(Orientation)


@dataclass(frozen=True, slots=True)
class Vehicle:
    id: VehicleId
    width: int
    length: int
    shipping_fee: int
    weight: int
    eta: int


@dataclass(frozen=True, slots=True)
class Ship:
    width: int
    length: int
    max_capacity: int


@dataclass(frozen=True, slots=True)
class Placement:
    x: int
    y: int
    orientation: Orientation


@dataclass(frozen=True, slots=True)
class Change:
    vehicle_id: VehicleId
    before: Placement | None
    after: Placement | None


type Move = tuple[Change, ...]
```

```python
type Objective = Callable[[State, Problem], float]


@dataclass(frozen=True, slots=True)
class State:
    placements: Mapping[VehicleId, Placement | None]

    def apply(self, move: Move) -> State:
        updated = dict(self.placements)
        for change in move:
            if change.vehicle_id not in self.placements:
                raise ValueError(f"kendaraan tidak dikenal:
{change.vehicle_id}")
            if self.placements[change.vehicle_id] != change.before:
                raise ValueError(f"placement awal {change.vehicle_id} tidak
cocok dengan move")
            updated[change.vehicle_id] = change.after
        return State(updated)

    def diff(self, other: State) -> Move:
        if self.placements.keys() != other.placements.keys():
            raise ValueError("kedua state harus memuat kendaraan yang sama")
        return tuple(
            Change(vehicle_id, self.placements[vehicle_id],
other.placements[vehicle_id])
            for vehicle_id in sorted(self.placements)
            if self.placements[vehicle_id] != other.placements[vehicle_id]
        )


@dataclass(frozen=True, slots=True)
class Problem:
    ship: Ship
    vehicles: tuple[Vehicle, ...]
    objective_name: str
    objective: Objective
```

```
    vehicle_by_id: Mapping[VehicleId, Vehicle] = field(init=False,
repr=False, compare=False)

    def __post_init__(self) -> None:
        if not self.vehicles:
            raise ValueError("problem harus memiliki minimal satu
kendaraan")
        vehicle_by_id = {vehicle.id: vehicle for vehicle in self.vehicles}
        if len(vehicle_by_id) != len(self.vehicles):
            raise ValueError("id kendaraan harus unik")
        object.__setattr__(self, "vehicle_by_id", vehicle_by_id)

    def value(self, state: State) -> float:
        return self.objective(state, self)
```

1.2. Prinsip *Feasible-Only* dan Pengecekan *Constraint*

Sistem bekerja **hanya dengan *state* yang valid** (*feasible-only*). Tidak ada penalti, sehingga sebuah *move* yang menghasilkan *state* yang melanggar batasan dibuang sebelum dipilih. Akibatnya algoritma tidak perlu mengurus pelanggaran, dan seluruh pengecekan batasan berada di satu modul (*core/constraints.py*).

1.3. Fungsi Objektif

```
# src/core/objectives.py
def total_shipping_fee(state: State, problem: Problem) -> float:
    return float(sum(vehicle.shipping_fee for vehicle in
loaded_vehicles(state, problem)))


def urgency_weighted_fee(state: State, problem: Problem) -> float:
    carried = loaded_vehicles(state, problem)
    return sum((vehicle.shipping_fee / vehicle.eta for vehicle in carried),
0.0)
```

```
OBJECTIVES: Final[dict[str, Objective]] = {
    "total-shipping-fee": total_shipping_fee,
    "urgency-weighted-fee": urgency_weighted_fee,
}
```

`total-shipping-fee` (bawaan) sesuai spesifikasi dan `urgency-weighted-fee` (bonus 1) memakai `shipping_fee / eta` tanpa menambah variabel baru. Semua algoritma memanggil `problem.value(state)` dan selalu memaksimalkan nilainya.

1.4. State Awal Acak

```
# src/core/initial.py
def random_state(problem: Problem, rng: random.Random) -> State:
    ship = problem.ship
    vehicles = list(problem.vehicles)
    rng.shuffle(vehicles)
    candidates = (
        (
            vehicle.id,
            Placement(
                rng.randrange(ship.width), rng.randrange(ship.length),
rng.choice(ORIENTATIONS)
            ),
        )
        for vehicle in vehicles
    )
    return place_in_order(problem, candidates)
```

Urutan kendaraan diacak, lalu tiap kendaraan dicoba ditempatkan pada letak acak, yang tidak muat, bertabrakan, atau membuat berat melebihi kapasitas dilewati (tetap di luar kapal). Dengan seed 1, 2, dan 3, state awal bernilai 441, 390, 349 (masing-masing berisi 8, 7, 6 kendaraan dari total 25 kendaraan).

1.5. Fungsi *Successor*

Ada dua cara memakai fungsi *successor*. `successors` mengembalikan seluruh tetangga valid dan dipakai *hill-climbing* yang membandingkan semuanya. `random_successor` mengembalikan satu tetangga valid acak dan dipakai *simulated annealing* serta mutasi pada *genetic algorithm*.

```
# src/core/neighbors.py
MAX_RANDOM_SUCCESSOR_ATTEMPTS: Final = 300


type Successor = tuple[Move, State]


def flipped(orientation: Orientation) -> Orientation:
    if orientation is Orientation.VERTICAL:
        return Orientation.HORIZONTAL
    return Orientation.VERTICAL


def rotation_move(vehicle: Vehicle, state: State) -> Move | None:
    placement = state.placements[vehicle.id]
    if placement is None or vehicle.width == vehicle.length:
        return None
    rotated = Placement(placement.x, placement.y,
flipped(placement.orientation))
    return (Change(vehicle.id, placement, rotated),)


def swap_move(first: Vehicle, second: Vehicle, state: State) -> Move | None:
    first_placement = state.placements[first.id]
    second_placement = state.placements[second.id]
    if first_placement == second_placement:
        return None
    return (
        Change(first.id, first_placement, second_placement),
        Change(second.id, second_placement, first_placement),
```

```
    )


def rotation_candidates(state: State, problem: Problem) -> Iterator[Move]:
    return filter(None, (rotation_move(vehicle, state) for vehicle in
problem.vehicles))


def relocation_candidates(state: State, problem: Problem) -> Iterator[Move]:
    ship = problem.ship
    for vehicle in problem.vehicles:
        current = state.placements[vehicle.id]
        if current is None:
            orientations = ORIENTATIONS
        else:
            orientations = (current.orientation,)
            yield (Change(vehicle.id, current, None),)
        for orientation in orientations:
            for x, y in deck_positions(vehicle, orientation, ship):
                target = Placement(x, y, orientation)
                if target != current:
                    yield (Change(vehicle.id, current, target),)


def swap_candidates(state: State, problem: Problem) -> Iterator[Move]:
    pairs = combinations(problem.vehicles, 2)
    return filter(None, (swap_move(first, second, state) for first, second
in pairs))


def successors(state: State, problem: Problem) -> Iterator[Successor]:
    occupancy = occupancy_of(state, problem)
    candidates = chain(
        rotation_candidates(state, problem),
        relocation_candidates(state, problem),
        swap_candidates(state, problem),
    )
```

```
    for move in candidates:
        if is_valid_move(move, occupancy, problem):
            yield move, state.apply(move)


type Sampler = Callable[[State, Problem, random.Random], Move | None]


def sample_rotation(state: State, problem: Problem, rng: random.Random) ->
Move | None:
    return rotation_move(rng.choice(problem.vehicles), state)


def sample_relocation(state: State, problem: Problem, rng: random.Random) ->
Move | None:
    ship = problem.ship
    vehicle = rng.choice(problem.vehicles)
    current = state.placements[vehicle.id]
    if current is None:
        orientation = rng.choice(ORIENTATIONS)
    else:
        orientation = current.orientation
        if rng.randrange(ship.width * ship.length + 1) == 0:
            return (Change(vehicle.id, current, None),)
    target = Placement(rng.randrange(ship.width),
rng.randrange(ship.length), orientation)
    if target == current:
        return None
    return (Change(vehicle.id, current, target),)


def sample_swap(state: State, problem: Problem, rng: random.Random) -> Move
| None:
    first, second = rng.sample(problem.vehicles, 2)
    return swap_move(first, second, state)
```

```python
SAMPLERS: Final[tuple[Sampler, ...]] = (sample_rotation, sample_relocation,
sample_swap)


def random_successor(state: State, problem: Problem, rng: random.Random) ->
Successor | None:
    vehicle_count = len(problem.vehicles)
    positions = problem.ship.width * problem.ship.length
    weights = (
        vehicle_count,
        vehicle_count * (positions + 1),
        vehicle_count * (vehicle_count - 1) // 2,
    )
    occupancy = occupancy_of(state, problem)
    for _ in range(MAX_RANDOM_SUCCESSOR_ATTEMPTS):
        sampler = rng.choices(SAMPLERS, weights=weights)[0]
        move = sampler(state, problem, rng)
        if move is not None and is_valid_move(move, occupancy, problem):
            return move, state.apply(move)
    valid = list(successors(state, problem))
    return rng.choice(valid) if valid else None
```

2. *Hill-Climbing*

Untuk setiap jenis *hill-climbing algorithm*, ada beberapa fungsi yang dipakai ulang.

```python
def scored_successors(state: State, problem: Problem) -> list[Candidate]:
    """
    Menghasilkan dan mengevaluasi seluruh kandidat tetangga (successors)
dari kondisi saat ini.

    Returns:
        List berisi tuple Candidate: (langkah/move, kondisi_tetangga,
nilai_evaluasi).
    """
```

```
    return [
        (move, successor, problem.value(successor))
        for move, successor in successors(state, problem)
    ]


def best_candidates(candidates: list[Candidate]) -> tuple[float,
list[Candidate]]:
    """
    Mencari nilai evaluasi tertinggi dari seluruh kandidat tetangga,
    serta mengumpulkan semua kandidat yang memiliki nilai maksimum tersebut
    (untuk menangani skenario seri/ties).
    """

    best_value = max(candidate[2] for candidate in candidates)
    return best_value, [candidate for candidate in candidates if
candidate[2] == best_value]


def choose_steepest(
    current_value: float, candidates: list[Candidate], rng: random.Random
) -> Candidate | None:

    if not candidates:
        return None
    best_value, best = best_candidates(candidates)
    # Pilih acak di antara kandidat terbaik jika ada perbaikan; jika tidak,
hentikan pencarian (return None)
    # Pindah hanya jika kandidat terbaik LEBIH BAIK dari kondisi saat ini.
    return rng.choice(best) if best_value > current_value else None


def choose_stochastic(
    current_value: float, candidates: list[Candidate], rng: random.Random
) -> Candidate | None:
```

```
    # Filter kandidat yang memberikan peningkatan dibanding nilai saat ini.
    improving = [candidate for candidate in candidates if candidate[2] >
current_value]
    # Pilih satu kandidat acak jika ada yang membaik. Jika tidak ada,
hentikan (return None)
    return rng.choice(improving) if improving else None


def climb(problem: Problem, rng: random.Random, choose: Choice) ->
SearchResult:

    # Memilih kondisi awal acak dan menghitung skor awalnya
    current = random_state(problem, rng)
    initial_state = current
    current_value = problem.value(current)

    evaluations = 1  # Menyimpan total berapa kali fungsi evaluasi
dipanggil
    history = [HistoryRecord(iteration=0, value=current_value)]  # Histori
nilai per iterasi
    steps: list[Step] = []
    iteration = 0

    while True:
        # Mengevaluasi seluruh kandidat tetangga dari kondisi saat ini
        candidates = scored_successors(current, problem)
        evaluations += len(candidates)

        chosen = choose(current_value, candidates, rng)
        if chosen is None:
            Break

        # Meng-update state saat ini dengan suksesor terpilih
        move, current, current_value = chosen
        iteration += 1
```

```
        # Catat histori
        steps.append(Step(iteration=iteration, changes=move))
        history.append(HistoryRecord(iteration=iteration,
value=current_value))
    return SearchResult(
        initial_state=initial_state,
        final_state=current,
        iterations=iteration,
        evaluations=evaluations,
        history=tuple(history),
        steps=tuple(steps),
        extra={},
    )
```

1.1. Steepest Ascent Hill-Climbing

```
# src/algorithms/hill_climbing.py
@dataclass(frozen=True, slots=True)
class SteepestAscent:
    def run(self, problem: Problem, rng: random.Random) -> SearchResult:
        return climb(problem, rng, choose_steepest)
```

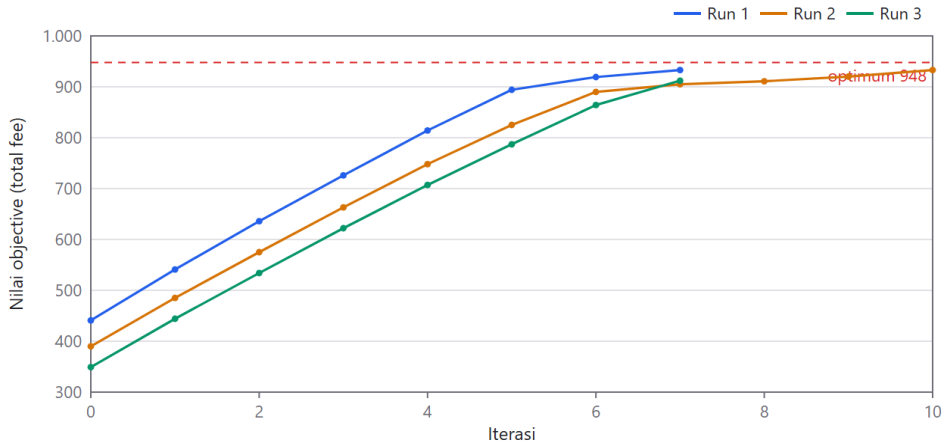
`SteepestAscent` didefinisikan sebagai sebuah class yang menjalan (*`run`*) fungsi `climb` dengan parameter berupa :

- `problem`: untuk membuat state dan melakukan pengujian.
- `rng` : mengatur kerandoman dalam pemilihan.
- `choose_steepest` : menentukan algoritma yang digunakan, dalam kasus ini menggunakan *steepest-ascent*.

Percobaan	*State* Awal	*State* Akhir	Nilai *Objective Function* Akhir	Durasi	Banyak Iterasi
1	*value state* 441, 8 kendaraan, berat 35/72	*value state* 933, 13 kendaraan, berat 72/72	933	0,228	7
2	*value state* 390, 7 kendaraan, berat 34/72	*value state* 933, 13 kendaraan, berat 72/72	933	0,219	10
3	*value state* 349, 6 kendaraan, berat 30/72	*value state* 912, 13 kendaraan, berat 72/72	912	0,199	7

Rata-rata nilai akhir 926,0 (simpangan baku 12,1, terendah 912, tertinggi 933), yaitu 97,7% dari optimum 948. Durasi rata-rata 0,215 detik.





1.2. Stochastic Hill-Climbing

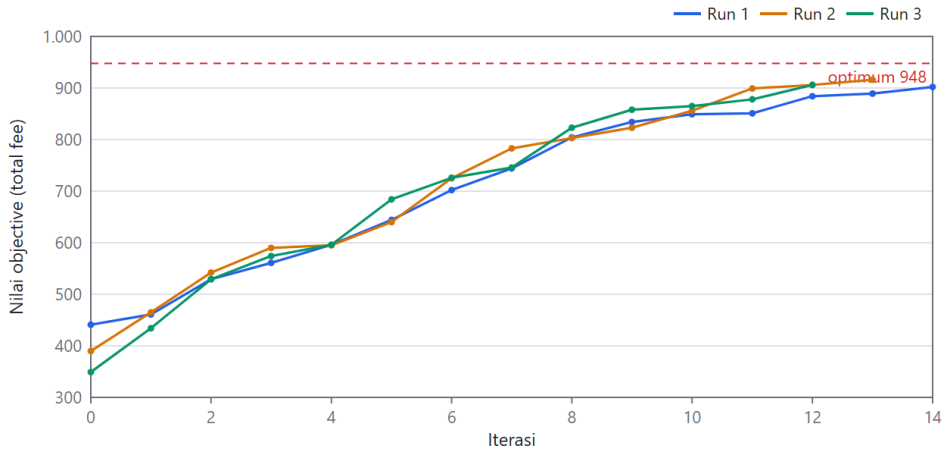
```python
# src/algorithms/hill_climbing.py
@dataclass(frozen=True, slots=True)
class SteepestAscent:
    def run(self, problem: Problem, rng: random.Random) -> SearchResult:
        return climb(problem, rng, choose_steepest)
```

`Stochastic` menjalankan `climb` dengan pemilih `choose_stochastic`, tidak memilih yang paling curam, melainkan satu langkah acak di antara semua langkah yang menaikkan nilai. Akibatnya langkahnya lebih kecil dan lebih banyak.

Percobaan	*State* Awal	*State* Akhir	Nilai *Objective Function* Akhir	Durasi	Banyak Iterasi
1	*value state* 441, 8 kendaraan, berat 35/72	*value state* 902, 15 kendaraan, berat 72/72	902	0,503	14
2	*value state* 390, 7 kendaraan, berat 34/72	*value state* 916, 14 kendaraan, berat 72/72	916	0,412	13
3	*value state* 349, 6 kendaraan, berat 30/72	*value state* 906, 15 kendaraan, berat 72/72	906	0,249	12

Rata-rata nilai akhir 908,0 (simpangan baku 7,2; terendah 902, tertinggi 916), yaitu 95,8% dari optimum 948. Durasi rata-rata 0,388 detik.





1.3. Hill-Climbing dengan Sideways Move

```
# src/algorithms/hill_climbing.py
def sideways_chooser(max_sideways: int) -> Choice:
    sideways_streak = 0

    def choose(
        current_value: float, candidates: list[Candidate], rng:
random.Random
    ) -> Candidate | None:
        nonlocal sideways_streak
        if not candidates:
            return None
        best_value, best = best_candidates(candidates)
```

```
        if best_value < current_value:
            return None
        if best_value == current_value:
            if sideways_streak >= max_sideways:
                return None
            sideways_streak += 1
        else:
            sideways_streak = 0
        return rng.choice(best)

    return choose


@dataclass(frozen=True, slots=True)
class SidewaysMove:
    max_sideways: int = 100

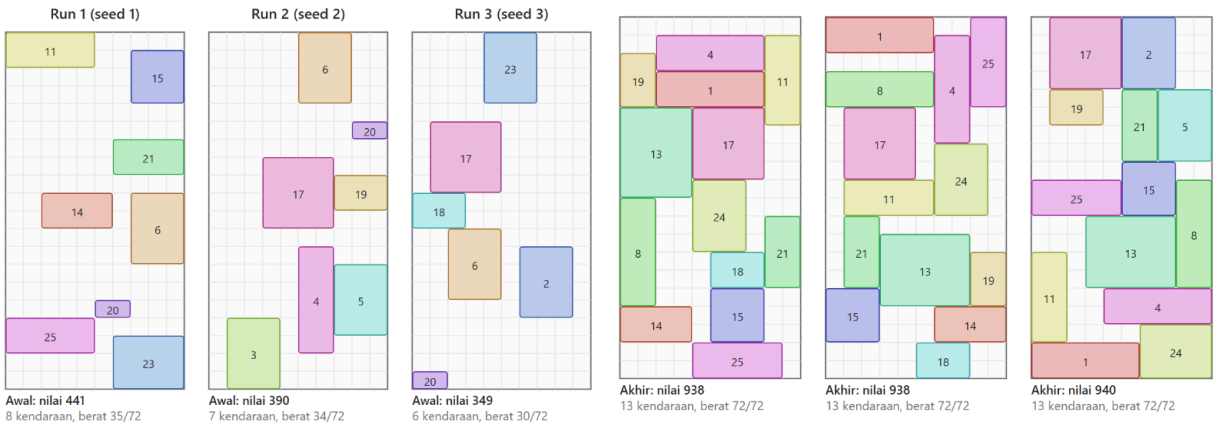
    def __post_init__(self) -> None:
        if self.max_sideways < 0:
            raise ValueError("max_sideways harus non-negatif")

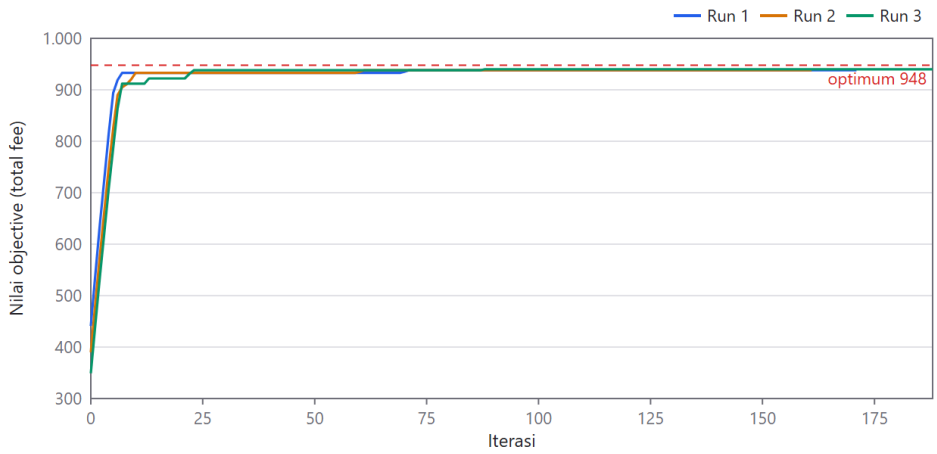
    def run(self, problem: Problem, rng: random.Random) -> SearchResult:
        result = climb(problem, rng, sideways_chooser(self.max_sideways))
        values = [record.value for record in result.history]
        sideways_used = sum(1 for before, after in pairwise(values) if
before == after)
        return replace(result, extra={"sideways_used":
float(sideways_used)})
```

`SideWaysMove` didefinisikan sebagai sebuah class yang menjalan (*run*) fungsi `climb` yang mirip dengan `SteepestAscent`. Parameter yang berbeda adalah `choose` yang menjalankan `sideways_chooser`, yaitu algoritma pencarian sideways-move.

Percobaan	*State* Awal	*State* Akhir	Nilai *Objective Function* Akhir	Durasi	Banyak Iterasi	Maximum Sideways Move	Langkah Sideways Dipakai
1	*value state* 441, 8 kendaraan, berat 35/72	*value state* 938, 13 kendaraan, berat 72/72	938	2,162	171	100	162
2	*value state* 390, 7 kendaraan, berat 34/72	*value state* 938, 13 kendaraan, berat 72/72	938	2,042	161	100	149
3	*value state* 349, 6 kendaraan, berat 30/72	*value state* 940, 13 kendaraan, berat 72/72	940	2,364	188	100	177

Rata-rata nilai akhir 938,7 (simpangan baku 1,2; terendah 938, tertinggi 940), yaitu 99,0% dari optimum 948. Durasi rata-rata 2,189 detik.





1.4. Random Restart Hill-Climbing

```
# src/algorithms/hill_climbing.py
def stitch_trajectories(
    results: list[SearchResult], best: SearchResult
) -> tuple[list[HistoryRecord], list[Step]]:
    history = list(results[0].history)
    steps = list(results[0].steps)
    reached = results[0].final_state
    for result in results[1:]:
        offset = history[-1].iteration + 1
        steps.append(Step(iteration=offset,
changes=reached.diff(result.initial_state)))
        history.extend(
            replace(record, iteration=record.iteration + offset) for record
in result.history
        )
        steps.extend(replace(step, iteration=step.iteration + offset) for
step in result.steps)
        reached = result.final_state
    if reached != best.final_state:
        iteration = history[-1].iteration + 1
        steps.append(Step(iteration=iteration,
changes=reached.diff(best.final_state)))
        history.append(HistoryRecord(iteration=iteration,
```

```
value=best.history[-1].value))
    return history, steps


@dataclass(frozen=True, slots=True)
class RandomRestart:
    inner: Algorithm = field(default_factory=SteepestAscent)
    max_restart: int = 10

    def __post_init__(self) -> None:
        if self.max_restart < 1:
            raise ValueError("max_restart harus minimal 1")

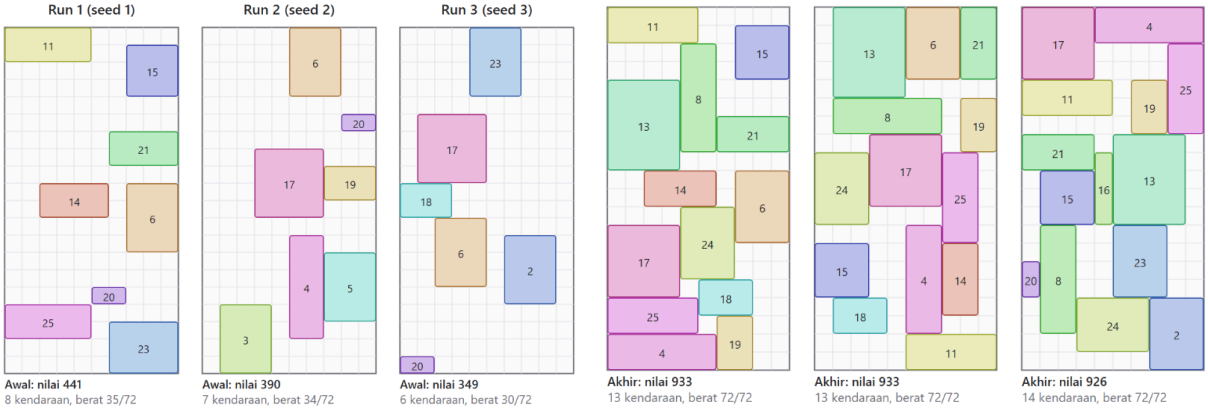
    def run(self, problem: Problem, rng: random.Random) -> SearchResult:
        results = [self.inner.run(problem, rng) for _ in
range(self.max_restart)]
        best_index = max(range(len(results)), key=lambda index:
results[index].history[-1].value)
        best = results[best_index]
        history, steps = stitch_trajectories(results, best)
        starts = accumulate((result.iterations + 1 for result in
results[:-1]), initial=0)
        extra = {
            "restarts": float(self.max_restart),
            "best_restart": float(best_index),
            **{
                f"start_iteration_restart_{index}": float(start)
                for index, start in enumerate(starts)
            },
            **{
                f"iterations_restart_{index}": float(result.iterations)
                for index, result in enumerate(results)
            },
        }
        return SearchResult(
            initial_state=results[0].initial_state,
            final_state=best.final_state,
```

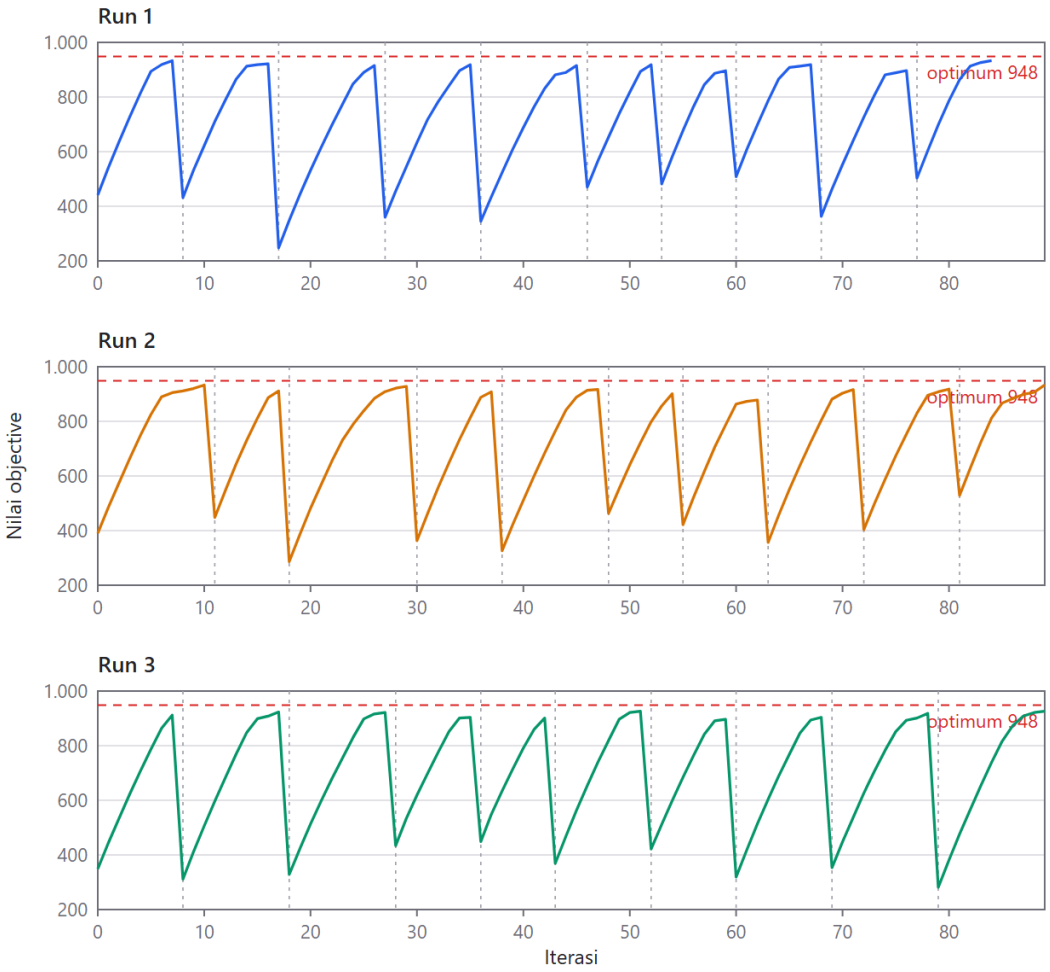
```
            iterations=history[-1].iteration,
            evaluations=sum(result.evaluations for result in results),
            history=tuple(history),
            steps=tuple(steps),
            extra=extra,
            unit=results[0].unit,
        )
```

Intinya, `RandomRestart` didefinisikan sebagai sebuah class yang menjalan (`run`) fungsi `self.inner.run`. Karena `RandomRestart` menggunakan `SteepestAscent` sebagai algoritma pencariannya, `climb` dilakukan `SteepestAscent`.

Percobaan	*State* Awal	*State* Akhir	Nilai *Objective Function* Akhir	Durasi	Banyak Iterasi	Maximum Restart	Banyak Restart	Restart Terbaik
1	*value state* 441, 8 kendaraan, berat 35/72	*value state* 933, 13 kendaraan, berat 72/72	933	1,524	84	10	10	0
2	*value state* 390, 7 kendaraan, berat 34/72	*value state* 938, 13 kendaraan, berat 72/72	938	2,042	161	10	10	0
3	*value state* 349, 6 kendaraan, berat 30/72	*value state* 940, 13 kendaraan, berat 72/72	940	2,364	188	10	10	5

Rata-rata nilai akhir 930,7 (simpangan baku 4,0; terendah 926, tertinggi 933), yaitu 98,2% dari optimum 948. Durasi rata-rata 1,753 detik.


Run 1 (seed 1)
Run 2 (seed 2)
Run 3 (seed 3)
Awal: nilai 441
8 kendaraan, berat 35/72
Awal: nilai 390
7 kendaraan, berat 34/72
Awal: nilai 349
6 kendaraan, berat 30/72
Akhir: nilai 933
13 kendaraan, berat 72/72
Akhir: nilai 933
13 kendaraan, berat 72/72
Akhir: nilai 926
14 kendaraan, berat 72/72


Run 1
Run 2
Run 3
optimum 948
Nilai objective
Iterasi
1.000
800
600
400
200
0
10
20
30
40
50
60
70
80

3. *Simmulated Annealing*

Algoritma *Simulated Annealing* diimplementasikan dalam `simulated_annealing.py`. Pada tiap iterasi, algoritma mengambil satu tetangga acak. Langkah yang memiliki nilai lebih baik dapat diterima dan langkah yang kurang baik juga masih dapat diterima dengan probabilitas `e^(△E/T)`, dengan ΔE adalah perubahan nilai dan T sebagai suhu. Hal ini bertujuan untuk memberikan kesempatan bagi pencarian untuk dapat keluar dari optimum lokal.

```
# src/algorithms/hill_climbing.py
def stitch_trajectories(
    results: list[SearchResult], best: SearchResult
) -> tuple[list[HistoryRecord], list[Step]]:
    history = list(results[0].history)
    steps = list(results[0].steps)
    reached = results[0].final_state
    for result in results[1:]:
        offset = history[-1].iteration + 1
        steps.append(Step(iteration=offset,
changes=reached.diff(result.initial_state)))
        history.extend(
            replace(record, iteration=record.iteration + offset) for record
in result.history
        )
        steps.extend(replace(step, iteration=step.iteration + offset) for
step in result.steps)
        reached = result.final_state
    if reached != best.final_state:
        iteration = history[-1].iteration + 1
        steps.append(Step(iteration=iteration,
changes=reached.diff(best.final_state)))
        history.append(HistoryRecord(iteration=iteration,
value=best.history[-1].value))
    return history, steps


@dataclass(frozen=True, slots=True)
```

```python
class RandomRestart:
    inner: Algorithm = field(default_factory=SteepestAscent)
    max_restart: int = 10

    def __post_init__(self) -> None:
        if self.max_restart < 1:
            raise ValueError("max_restart harus minimal 1")

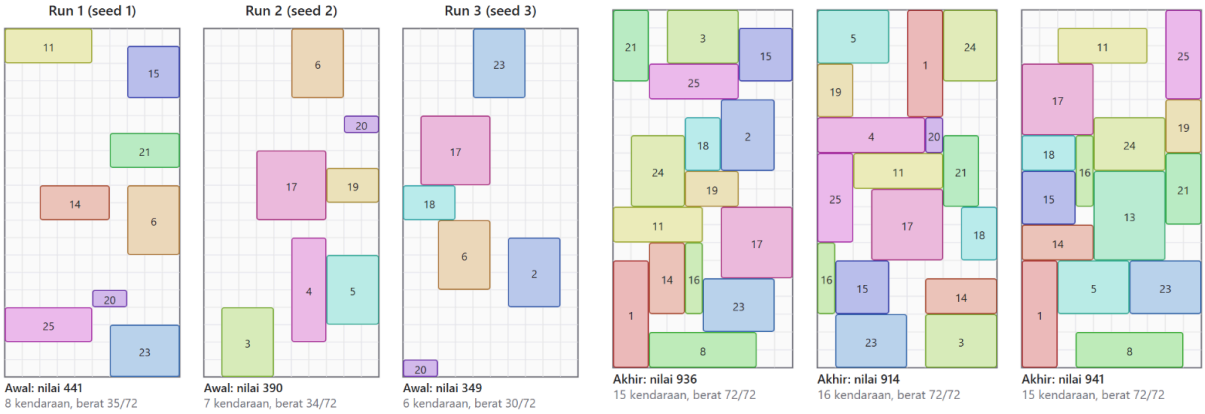
    def run(self, problem: Problem, rng: random.Random) -> SearchResult:
        results = [self.inner.run(problem, rng) for _ in
range(self.max_restart)]
        best_index = max(range(len(results)), key=lambda index:
results[index].history[-1].value)
        best = results[best_index]
        history, steps = stitch_trajectories(results, best)
        starts = accumulate((result.iterations + 1 for result in
results[:-1]), initial=0)
        extra = {
            "restarts": float(self.max_restart),
            "best_restart": float(best_index),
            **{
                f"start_iteration_restart_{index}": float(start)
                for index, start in enumerate(starts)
            },
            **{
                f"iterations_restart_{index}": float(result.iterations)
                for index, result in enumerate(results)
            },
        }
        return SearchResult(
            initial_state=results[0].initial_state,
            final_state=best.final_state,
            iterations=history[-1].iteration,
            evaluations=sum(result.evaluations for result in results),
            history=tuple(history),
            steps=tuple(steps),
            extra=extra,
```

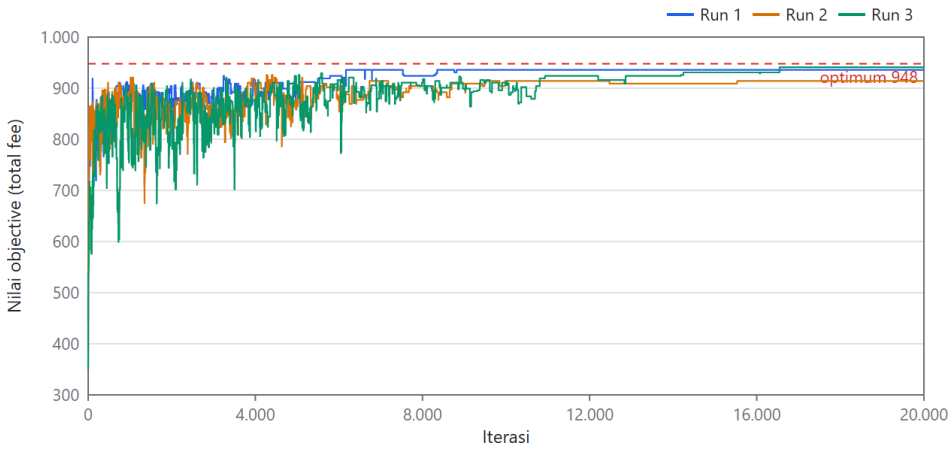
```
            unit=results[0].unit,
        )
```

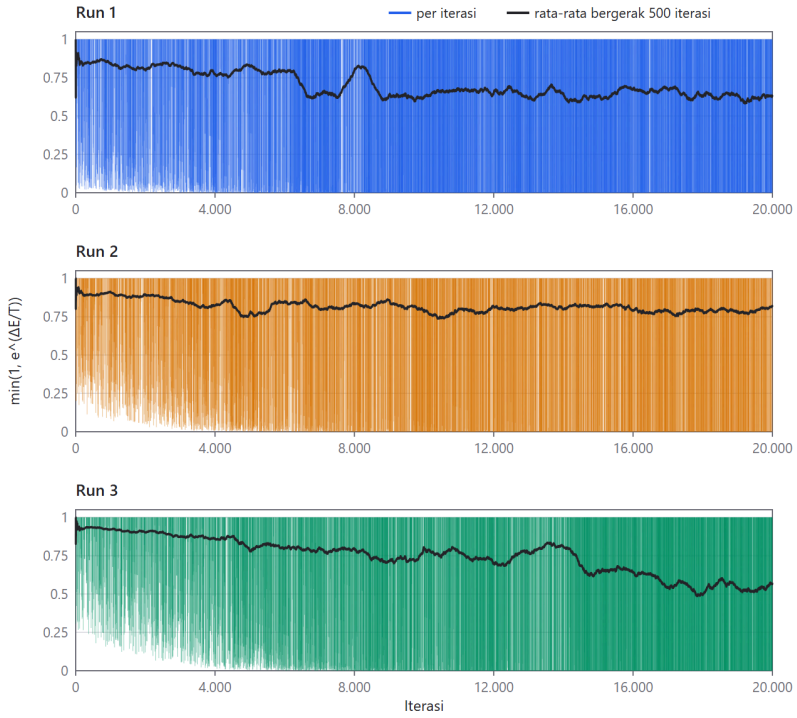
Kelas ini menjalankan proses utama dengan metode `run`. Metode ini membuat *state* awal acak, menghitung nilainya, kemudian berulang kali meminta nilai tetangga acak yang valid melalui `random_successor`. Jika nilai tetangganya lebih baik maka akan diterima, jika tidak maka algoritma akan menggunakan peluang penerimaan yang sudah dihitung. Setelah tiap iterasi, suhu diturunkan. Proses berhenti ketika jumlah iterasi sudah mencapai batas, suhu turun dibawah batas minimum, atau tidak ada tetangga yang valid lagi

Percobaan	*State* Awal	*State* Akhir	Nilai *Objective Function* Akhir	Durasi	Banyak Iterasi	Frekuensi 'stuck' di lokal optima
1	*value state* 441, 8 kendaraan, berat 35/72	*value state* 936, 15 kendaraan, berat 72/72	936	7,093	20.000	0,00040 (8 kejadian)
2	*value state* 390, 7 kendaraan, berat 34/72	*value state* 914, 16 kendaraan, berat 72/72	914	6,355	20.000	0,00045 (9 kejadian)
3	*value state* 349, 6 kendaraan, berat 30/72	*value state* 941, 15 kendaraan, berat 72/72	941	6,995	20.000	0,00035 (7 kejadian)

Rata-rata nilai akhir 930,3 (simpangan baku 14,4; terendah 914, tertinggi 941), yaitu 98,1% dari optimum 948. Durasi rata-rata 6,814 detik.


Run 1 (seed 1)
Run 2 (seed 2)
Run 3 (seed 3)
Awal: nilai 441
8 kendaraan, berat 35/72
Awal: nilai 390
7 kendaraan, berat 34/72
Awal: nilai 349
6 kendaraan, berat 30/72
Akhir: nilai 936
15 kendaraan, berat 72/72
Akhir: nilai 914
16 kendaraan, berat 72/72
Akhir: nilai 941
15 kendaraan, berat 72/72


Run 1
Run 2
Run 3
optimum 948
Nilai objective (total fee)
Iterasi
1.000
900
800
700
600
500
400
300
0
4.000
8.000
12.000
16.000
20.000


Run 1
per iterasi
rata-rata bergerak 500 iterasi
Run 2
Run 3
min(1, e^(ΔE/T))
Iterasi
1
0.75
0.5
0.25
0
0
4.000
8.000
12.000
16.000
20.000

4. *Genetic Algorithm*

Genetic Algorithm menjalankan pencarian terhadap sekumpulan solusi yang disebut populasi. Setiap individu adalah satu state penempatan kendaraan, dan nilai objektif state digunakan sebagai *fitness*. Pada tiap generasi, algoritma memilih induk, menggabungkan sebagian solusi mereka untuk membentuk anak, lalu terkadang memutasi anak tersebut. Beberapa individu terbaik dipertahankan agar solusi berkualitas tidak hilang.

```
# src/algorithms/genetic_algorithm.py
def roulette_weights(fitness: Sequence[float]) -> list[float] | None:
    offset = min(0.0, *fitness)
    cumulative_weights = list(accumulate(value - offset for value in
fitness))
    return cumulative_weights if cumulative_weights[-1] > 0 else None


def select_parent(
    population: Sequence[State], cumulative_weights: list[float] | None,
rng: random.Random
) -> State:
    if cumulative_weights is None:
        return rng.choice(population)
    return rng.choices(population, cum_weights=cumulative_weights)[0]


def crossover(first: State, second: State, problem: Problem, rng:
random.Random) -> State:
    crossover_point = rng.randrange(len(problem.vehicles) + 1)
    genes: list[tuple[VehicleId, Placement]] = []
    for index, vehicle in enumerate(problem.vehicles):
        placement = (first if index < crossover_point else
second).placements[vehicle.id]
        if placement is not None:
            genes.append((vehicle.id, placement))
    return place_in_order(problem, genes)
```

```
def mutate(state: State, problem: Problem, rng: random.Random) -> State:
    successor = random_successor(state, problem, rng)
    return state if successor is None else successor[1]


def elite_indices(fitness: Sequence[float], count: int) -> list[int]:
    ranked = sorted(range(len(fitness)), key=lambda index: fitness[index],
reverse=True)
    return ranked[:count]


def best_index(fitness: Sequence[float]) -> int:
    return max(range(len(fitness)), key=fitness.__getitem__)


def generation_record(generation: int, fitness: Sequence[float]) ->
HistoryRecord:
    return HistoryRecord(
        iteration=generation,
        value=max(fitness),
        metrics={"average_value": sum(fitness) / len(fitness)},
    )


@dataclass(frozen=True, slots=True)
class GeneticAlgorithm:
    population_size: int = 50
    generations: int = 100
    mutation_rate: float = 0.3
    elitism_count: int = 2

    def __post_init__(self) -> None:
        if self.population_size <= 0:
            raise ValueError("population_size harus positif")
        if self.generations < 0:
            raise ValueError("generations harus non-negatif")
```

```
        if not 0 <= self.mutation_rate <= 1:
            raise ValueError("mutation_rate harus berada di antara 0 dan 1")
        if not 0 <= self.elitism_count <= self.population_size:
            raise ValueError("elitism_count harus berada di antara 0 dan
population_size")

    def next_generation(
        self,
        population: Sequence[State],
        fitness: Sequence[float],
        problem: Problem,
        rng: random.Random,
    ) -> tuple[list[State], list[float]]:
        elites = elite_indices(fitness, self.elitism_count)
        next_population = [population[index] for index in elites]
        next_fitness = [fitness[index] for index in elites]
        cumulative_weights = roulette_weights(fitness)
        while len(next_population) < self.population_size:
            first_parent = select_parent(population, cumulative_weights,
rng)
            second_parent = select_parent(population, cumulative_weights,
rng)
            child = crossover(first_parent, second_parent, problem, rng)
            if rng.random() < self.mutation_rate:
                child = mutate(child, problem, rng)
            next_population.append(child)
            next_fitness.append(problem.value(child))
        return next_population, next_fitness

    def run(self, problem: Problem, rng: random.Random) -> SearchResult:
        population = [random_state(problem, rng) for _ in
range(self.population_size)]
        fitness = [problem.value(state) for state in population]
        evaluations = len(population)
        initial_best = population[best_index(fitness)]
        previous_best = initial_best
        history = [generation_record(0, fitness)]
```

```
        steps: list[Step] = []

        for generation in range(1, self.generations + 1):
            population, fitness = self.next_generation(population, fitness,
problem, rng)
            evaluations += self.population_size - self.elitism_count
            current_best = population[best_index(fitness)]
            history.append(generation_record(generation, fitness))
            steps.append(Step(generation, previous_best.diff(current_best)))
            previous_best = current_best

        return SearchResult(
            initial_state=initial_best,
            final_state=previous_best,
            iterations=self.generations,
            evaluations=evaluations,
            history=tuple(history),
            steps=tuple(steps),
            extra={},
            unit="generation",
        )
```

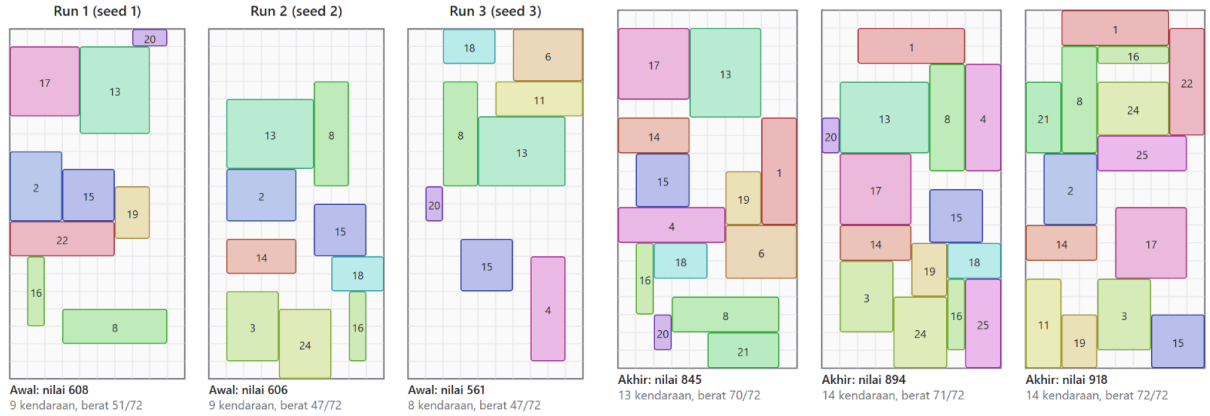
Kelas `GeneticAlgorithm` mengatur parameter ukuran populasi bawaan 50, jumlah generasi 100, peluang mutasi 0.3, dan jumlah individu elit bawaan 2. Metode `next_generation` menyalin individu elit, memilih pasangan induk, membuat anak crossover, dan melakukan mutasi sesuai peluangnya. Proses diulang sampai ukuran populasi berikutnya terpenuhi.

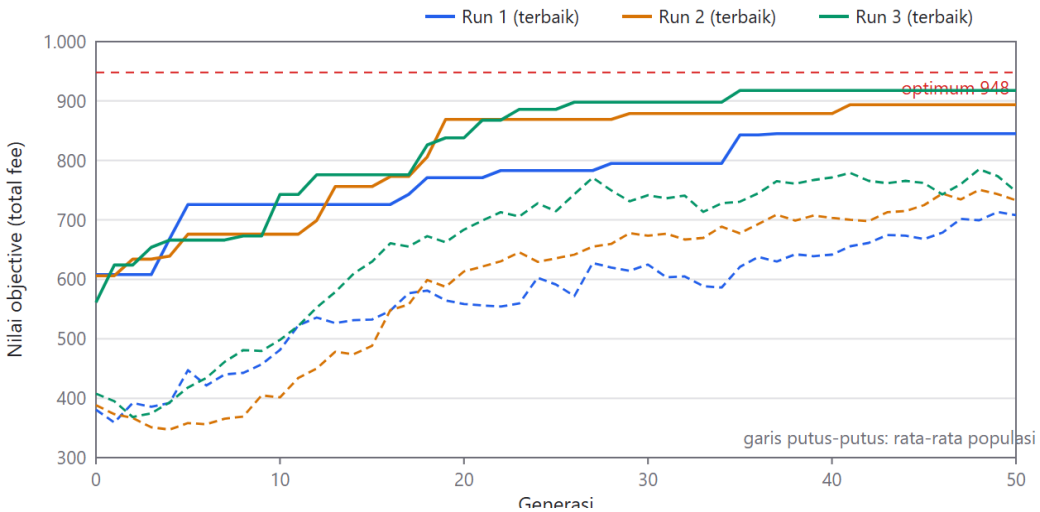
POPULASI 50, 50 GENERASI

Percobaan	*State* Awal	*State* Akhir	Nilai *Objective Function* Akhir	Durasi	Banyak Iterasi	Jumlah Populasi
1	*value state* 608, 9 kendaraan, berat 51/72	*value state* 845, 13 kendaraan, berat 70/72	845	0,256	50	50

Percobaan	*State* Awal	*State* Akhir	Nilai *Objective Function* Akhir	Durasi	Banyak Iterasi	Jumlah Populasi
2	*value state* 606, 9 kendaraan, berat 47/72	*value state* 894, 14 kendaraan, berat 71/72	894	0,273	50	50
3	*value state* 561, 8 kendaraan, berat 47/72	*value state* 918, 14 kendaraan, berat 72/72	918	0,290	50	50

Rata-rata nilai akhir 885,7 (simpangan baku 37,2; terendah 845, tertinggi 918), yaitu 93,4% dari optimum 948. Durasi rata-rata 0,273 detik.



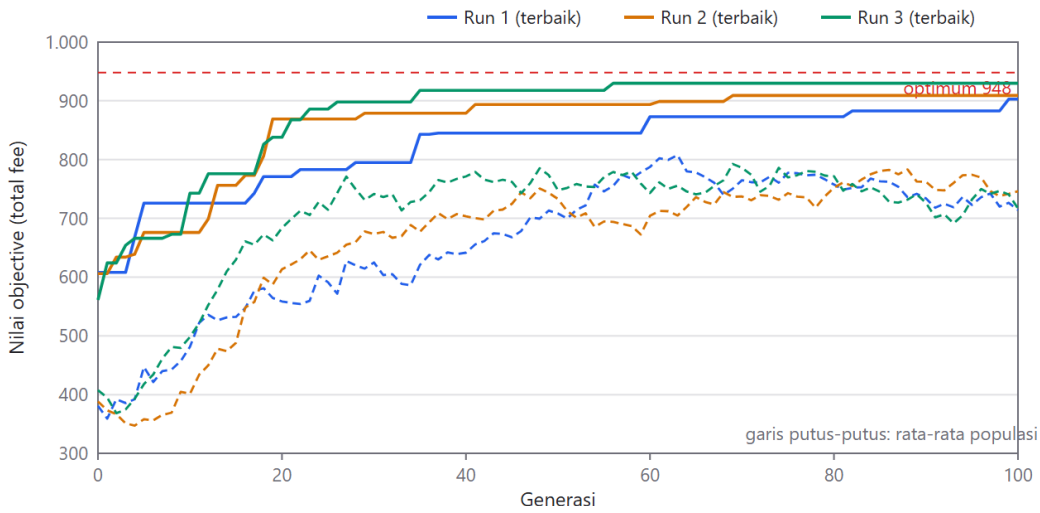


POPULASI 50, 100 GENERASI

Percobaan	*State* Awal	*State* Akhir	Nilai *Objective Function* Akhir	Durasi	Banyak Iterasi	Jumlah Populasi
1	*value state* 608, 9 kendaraan, berat 51/72	*value state* 903, 13 kendaraan, berat 70/72	903	0,605	100	50
2	*value state* 606, 9 kendaraan, berat 47/72	*value state* 909, 14 kendaraan, berat 71/72	909	0,660	100	50
3	*value state* 561, 8 kendaraan, berat 47/72	*value state* 930, 14 kendaraan, berat 72/72	930	0,620	100	50

Rata-rata nilai akhir 914,0 (simpangan baku 14,2; terendah 903, tertinggi 930), yaitu 96,4% dari optimum 948. Durasi rata-rata 0,628 detik.

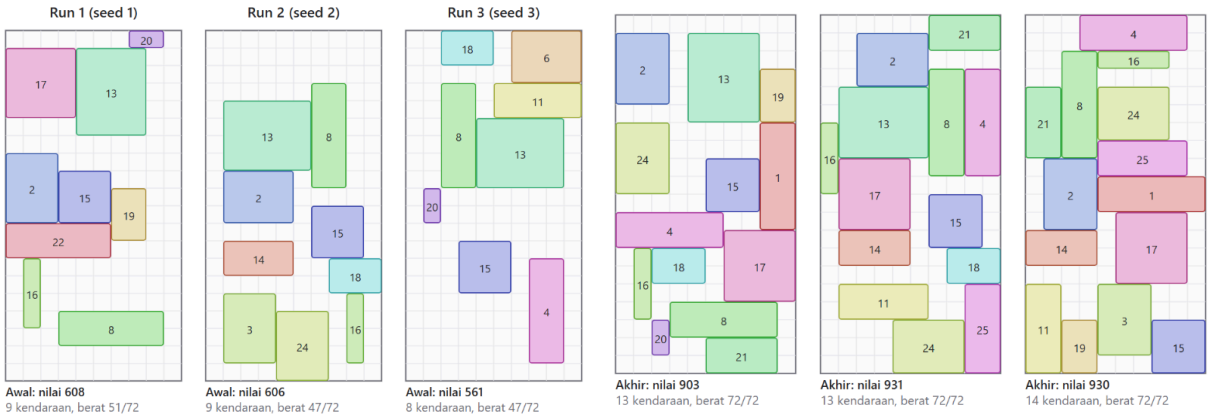


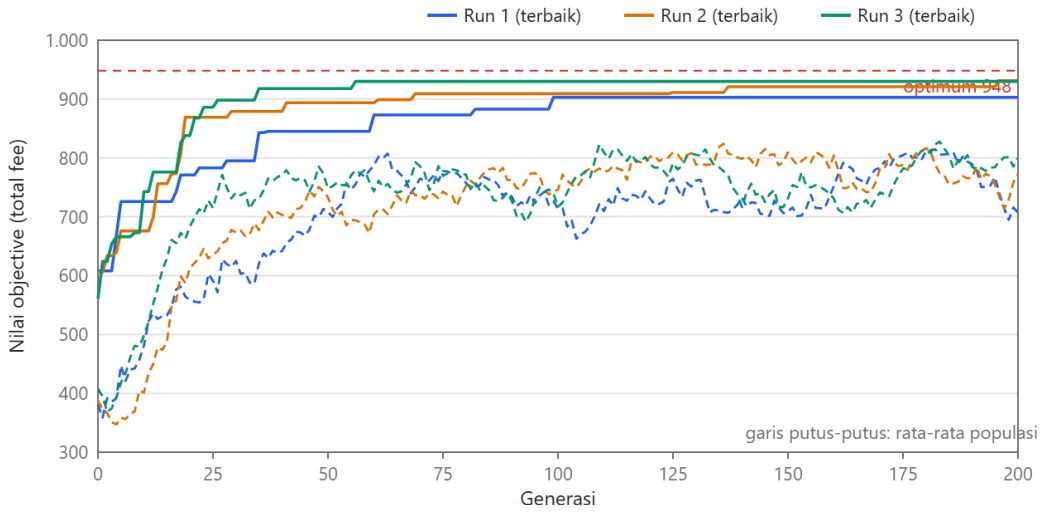


POPULASI 50, 200 GENERASI

Percobaan	*State* Awal	*State* Akhir	Nilai *Objective Function* Akhir	Durasi	Banyak Iterasi	Jumlah Populasi
1	*value state* 608, 9 kendaraan, berat 51/72	*value state* 903, 13 kendaraan, berat 70/72	903	1,688	200	50
2	*value state* 606, 9 kendaraan, berat 47/72	*value state* 931, 14 kendaraan, berat 71/72	931	1,375	200	50
3	*value state* 561, 8 kendaraan, berat 47/72	*value state* 930, 14 kendaraan, berat 72/72	930	1,362	200	50

Rata-rata nilai akhir 921,3 (simpangan baku 15,9; terendah 903, tertinggi 931), yaitu 97,2% dari optimum 948. Durasi rata-rata 1,475 detik.



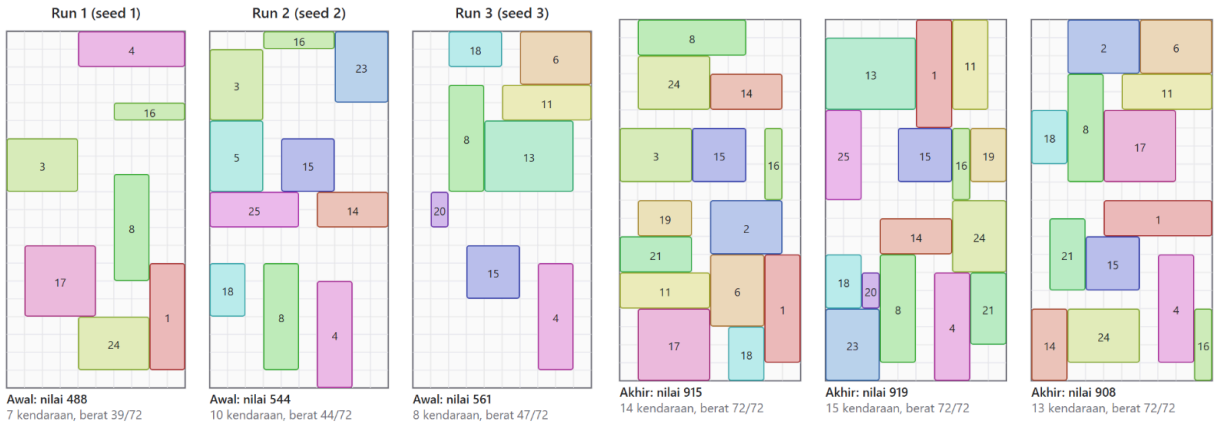


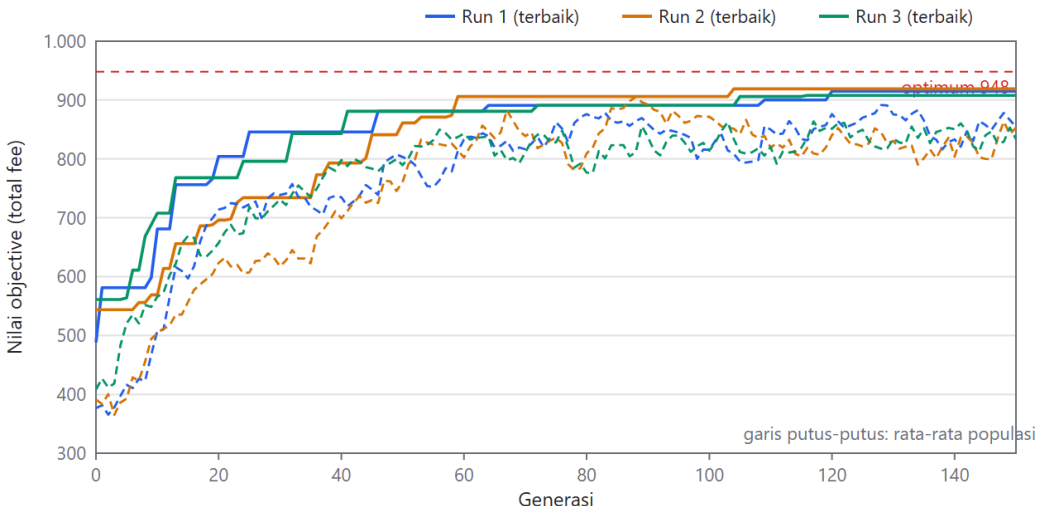
20 POPULASI, 150 GENERASI

Percobaan	*State* Awal	*State* Akhir	Nilai *Objective Function* Akhir	Durasi	Banyak Iterasi	Jumlah Populasi
1	*value state* 488, 9 kendaraan, berat 51/72	*value state* 915, 13 kendaraan, berat 70/72	915	0,400	150	20
2	*value state* 544, 9 kendaraan, berat 47/72	*value state* 919, 14 kendaraan, berat 71/72	919	0,383	150	20

Percobaan	*State* Awal	*State* Akhir	Nilai *Objective Function* Akhir	Durasi	Banyak Iterasi	Jumlah Populasi
3	*value state* 561, 8 kendaraan, berat 47/72	*value state* 908, 14 kendaraan, berat 72/72	908	0,368	150	20

Rata-rata nilai akhir 914,0 (simpangan baku 5,6; terendah 908, tertinggi 919), yaitu 96,4% dari optimum 948. Durasi rata-rata 0,384 detik.



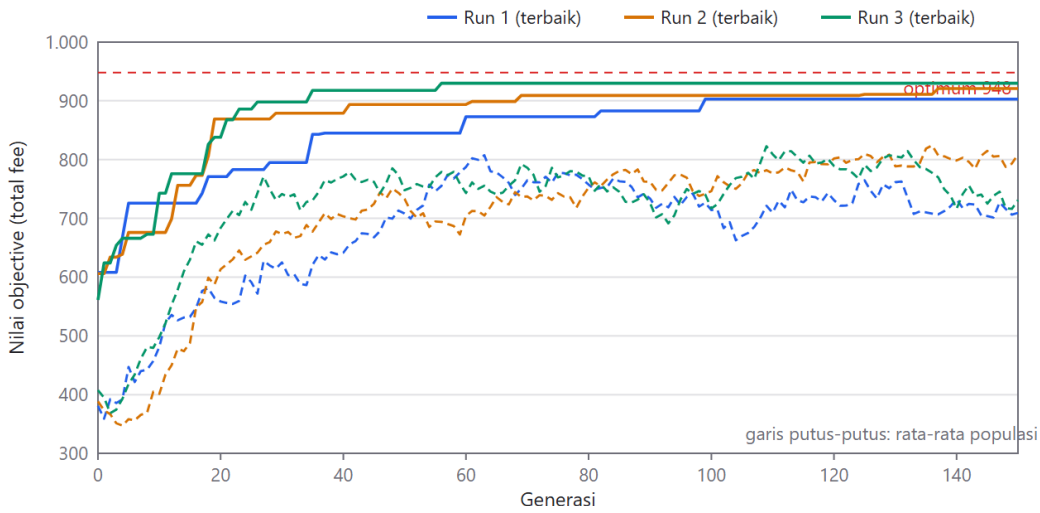


50 POPULASI, 150 GENERASI

Percobaan	*State* Awal	*State* Akhir	Nilai *Objective Function* Akhir	Durasi	Banyak Iterasi	Jumlah Populasi
1	*value state* 608, 9 kendaraan, berat 51/72	*value state* 903, 13 kendaraan, berat 70/72	903	0,918	150	50
2	*value state* 606, 9 kendaraan, berat 47/72	*value state* 921, 14 kendaraan, berat 71/72	921	0,956	150	50
3	*value state* 561, 8 kendaraan, berat 47/72	*value state* 930, 14 kendaraan, berat 72/72	930	0,998	150	50

Rata-rata nilai akhir 918,0 (simpangan baku 13,7; terendah 903, tertinggi 930), yaitu 96,8% dari optimum 948. Durasi rata-rata 0,957 detik.



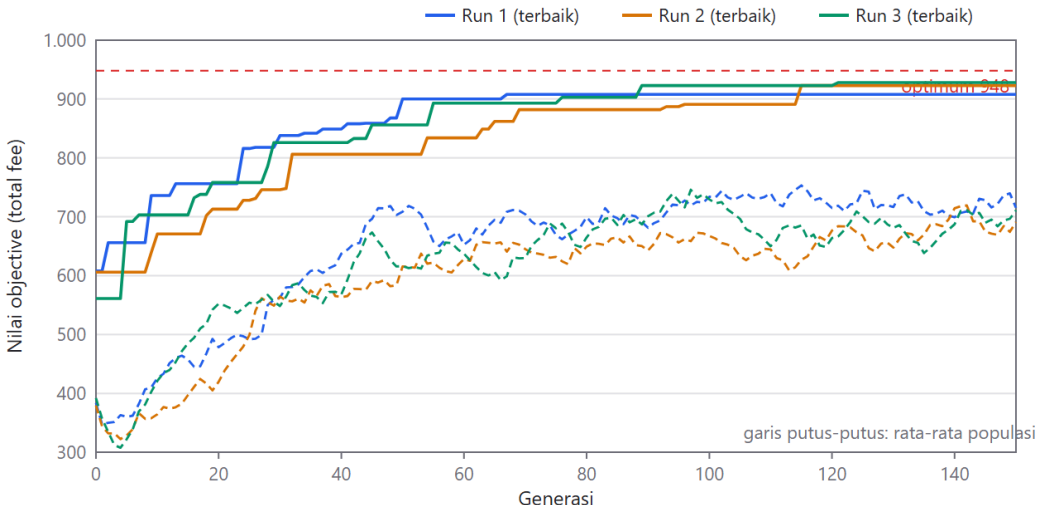


100 POPULASI, 150 GENERASI

Percobaan	*State* Awal	*State* Akhir	Nilai *Objective Function* Akhir	Durasi	Banyak Iterasi	Jumlah Populasi
1	*value state* 608, 9 kendaraan, berat 51/72	*value state* 908, 13 kendaraan, berat 70/72	908	1,786	150	100
2	*value state* 606, 9 kendaraan, berat 47/72	*value state* 923, 14 kendaraan, berat 71/72	923	1,698	150	100
3	*value state* 561, 8 kendaraan, berat 47/72	*value state* 928, 14 kendaraan, berat 72/72	928	1,778	150	100

Rata-rata nilai akhir 919,7 (simpangan baku 10,4; terendah 908, tertinggi 928), yaitu 97,0% dari optimum 948. Durasi rata-rata 1,754 detik.


Akhir: nilai 903
13 kendaraan, berat 72/72
Akhir: nilai 921
13 kendaraan, berat 72/72
Akhir: nilai 930
14 kendaraan, berat 72/72
Akhir: nilai 908
14 kendaraan, berat 72/72
Akhir: nilai 923
14 kendaraan, berat 72/72
Akhir: nilai 928
13 kendaraan, berat 72/72


Run 1 (terbaik)
Run 2 (terbaik)
Run 3 (terbaik)
Nilai objective (total fee)
1.000
900
800
700
600
500
400
300
0
20
40
60
80
100
120
140
Generasi
garis putus-putus: rata-rata populasi

BAB 4
ANALISIS DAN HASIL EKSPERIMEN

1. Ringkasan Hasil

Konfigurasi	**R1**	**R2**	**R3**	**Rerata**	**Simp. Baku**	**Min**	**Max**	**% optimum (rata-rata)**
Steepest-ascent	933	933	912	926,0	12,1	912	933	97,7%
Sideways	938	938	940	938,7	1,2	938	940	99,0%
Stochastic	902	916	906	908,0	7,2	902	916	95,8%
Random	933	933	926	930,7	4,0	926	933	98,2%
Simulated	936	914	941	930,3	14,4	914	941	98,1%
GA, 50 generasi (populasi 50)	845	894	918	885,7	37,2	845	918	93,4%
GA, 100 generasi (populasi 50)	903	909	930	914,0	14,2	903	930	96,4%
GA, 200 generasi (populasi 50)	903	931	930	921,3	15,9	903	931	97,2%
GA, populasi 20 (150 generasi)	915	919	908	914,0	5,6	908	919	96,4%
GA, populasi 50 (150 generasi)	903	921	930	918,0	13,7	903	930	96,8%
GA, populasi 100 (150 generasi)	908	923	928	919,7	10,4	908	928	97,0%

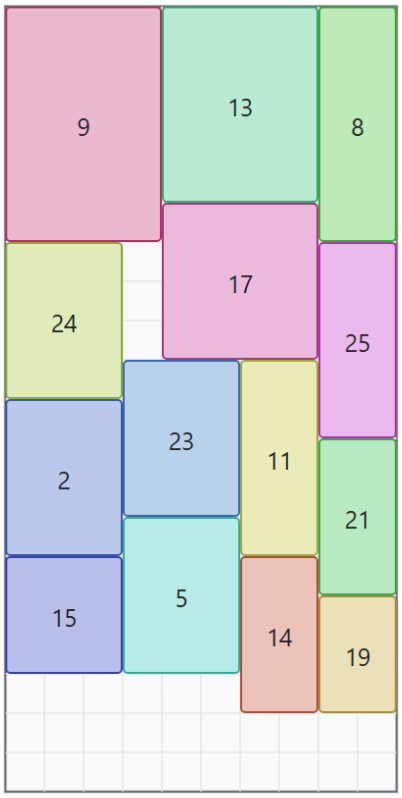
Konfigurasi	Durasi run 1; 2; 3 (detik)	Rata-rata (detik)	Iterasi (run 1; 2; 3)	Evaluasi objective (rata-rata)	Durasi per iterasi (ms)

Steepest-ascent	0,228; 0,219; 0,199	0,215	7; 10; 7	4.461	27,65
Sideways	2,162; 2,042; 2,364	2,189	171; 161; 188	25.310	12,63
Stochastic	0,503; 0,412; 0,249	0,388	14; 13; 12	6.675	29,46
Random	1,524; 1,666; 2,070	1,753	84; 89; 89	42.902	20,04
Simulated	7,093; 6,355; 6,995	6,814	20.000; 20.000; 20.000	20.101	0,34
GA, 50 generasi (populasi 50)	0,256; 0,273; 0,290	0,273	50; 50; 50	2.450	5,46
GA, 100 generasi (populasi 50)	0,605; 0,660; 0,620	0,628	100; 100; 100	4.850	6,28
GA, 200 generasi (populasi 50)	1,688; 1,375; 1,362	1,475	200; 200; 200	9.650	7,37
GA, populasi 20 (150 generasi)	0,400; 0,383; 0,368	0,384	150; 150; 150	2.720	2,56
GA, populasi 50 (150 generasi)	0,918; 0,956; 0,998	0,957	150; 150; 150	7.250	6,38
GA, populasi 100 (150	1,786; 1,698; 1,778	1,754	150; 150; 150	14.800	11,69

generasi)					

2. Seberapa Dekat dengan Optimum Global?

Optimum global instance ini adalah 948. Batas atasnya diperoleh dari knapsack berat (fee maksimum dengan berat ≤ 72, dihitung eksak) dan tetap 948 setelah batas luas ditambahkan. Batas itu tercapai: salah satu dari 8 himpunan kendaraan bernilai 948 (14 kendaraan, berat 72, luas 171) terbukti dapat disusun di geladak dan lolos pengecek constraint program.



Optimum: nilai 948
14 kendaraan, berat 72/72, luas 171/200

Tidak ada run yang mencapai 948 (0 dari 33). Hasil terbaik adalah 941 (simulated annealing, run 3), yaitu 99,26% dari optimum; rata-rata tiap konfigurasi 93,4% sampai 99,0% dari optimum.

Penyebabnya adalah struktur masalah. Pada 31 dari 33 state akhir berat terangkut tepat 72, sedangkan luas terpakai hanya 127 sampai 157 dari 200 sel; jadi batas berat yang menentukan, dan masalahnya mirip knapsack yang kendaraan pilihannya harus bisa disusun. Naik dari optimum lokal biasanya memerlukan menukar beberapa kendaraan

sekaligus dengan berat yang pas, padahal satu move hanya memutar, memindahkan, atau menukar dua kendaraan. Semua state akhir berbeda sedikitnya 3 kendaraan dari himpunan optimum terdekat (Lampiran A).

3. Perbandingan Hasil dan Konsistensi

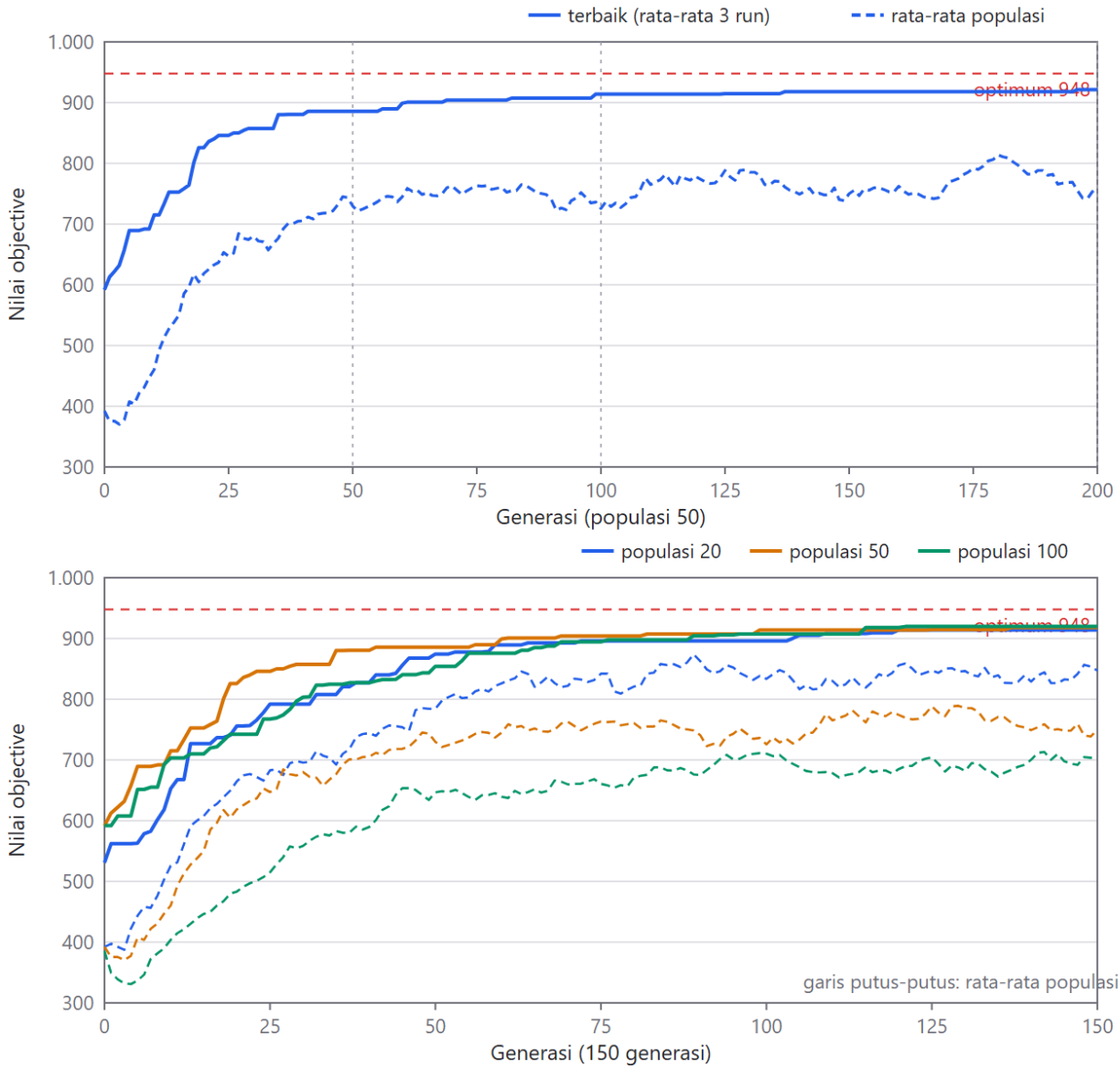
- Sideways-move terbaik dan paling konsisten (938,7; simpangan baku 1,2), lebih tinggi dari steepest-ascent pada ketiga seed. Ketiga run berhenti karena batas 100 langkah sideways tercapai (Bagian 2.3), sehingga max_sideways yang lebih besar layak dicoba.
- Steepest-ascent (926,0) dan stochastic (908,0) berhenti cepat di optimum lokal, stochastic lebih rendah dari steepest-ascent pada ketiga seed karena tidak memilih langkah paling curam.
- Random-restart (930,7) tidak pernah lebih buruk dari steepest-ascent pada seed yang sama (restart pertamanya identik) dengan biaya sekitar 9,6 kali evaluasi.
- Simulated annealing (930,3) menghasilkan nilai terbaik (941) tetapi juga 914, karena SA mengembalikan state saat berhenti (run 2 sempat melihat 926). Rata-ratanya tidak dapat dibedakan dari random-restart.
- Genetic algorithm (885,7 sampai 921,3) berada di bawah hill-climbing terbaik. Kemungkinan, crossover pada urutan kendaraan banyak membuang gen sehingga kemajuan terutama berasal dari mutasi dan elitisme.

Dengan hanya 3 run per konfigurasi, selisih kecil (misalnya random-restart melawan SA) tidak dapat disimpulkan signifikan. Pada ke-30 climb random-restart, rata-rata steepest-ascent adalah 913,3, sehingga angka 926,0 pada tiga seed kemungkinan sedikit terlalu optimis.

4. Perbandingan Durasi

Durasi dari tercepat: Steepest-ascent (0,22 detik) hingga Simulated annealing (6,81 detik); lihat Tabel 19 dan Gambar 29. Hill-climbing hanya butuh sedikit iterasi tetapi tiap iterasi memeriksa ribuan kandidat move (sekitar 5.400 pada state awal contoh), sekitar 28 ms per iterasi. Simulated annealing hanya membangkitkan satu tetangga per iterasi (0,34 ms) tetapi selalu menjalankan 20.000 iterasi, sehingga paling lama. Durasi GA kira-kira linear terhadap populasi × generasi. Durasi diukur pada satu mesin dan dipengaruhi beban sistem (stochastic: 0,249 sampai 0,503 detik), jadi yang dapat diandalkan adalah urutannya, bukan angka mutlak.

5. Pengaruh Banyak Iterasi dan Jumlah Populasi pada GA



- Banyak iterasi (populasi 50): rata-rata 885,7 → 914,0 → 921,3 untuk 50, 100, 200 generasi. Hasil bertambah tetapi kenaikannya mengecil, dan nilai terbaik tidak pernah turun karena elitisme.
- Jumlah populasi (150 generasi): rata-rata 914,0 → 918,0 → 919,7 untuk populasi 20, 50, 100. Kenaikan ini jauh di bawah simpangan baku sehingga pengaruhnya tidak terbukti, padahal evaluasi naik 5,4 kali.
- Karena seed sama, konfigurasi populasi 50 (50, 100, 150, 200 generasi) menempuh lintasan yang sama dan hanya berbeda panjang, sehingga tidak independen.

6. Keterbatasan

Hanya satu instance, 3 run per konfigurasi, tanpa uji signifikansi, dan durasi diukur pada satu mesin. Penjelasan sebab (mengapa sideways unggul, mengapa GA tertinggal) adalah dugaan yang konsisten dengan data tetapi belum diuji.

BAB 5
KESIMPULAN DAN SARAN

Kesimpulan:

1. Empat varian hill-climbing, simulated annealing, dan genetic algorithm berhasil diimplementasikan dan dijalankan 33 kali (11 konfigurasi × 3 run). Setiap state selalu valid dan hasil dapat diulang dengan seed yang sama.
2. Optimum global instance ini adalah 948. Hasil terbaik 941 (Simulated annealing run 3) mencapai 99,26% dari optimum; tidak ada run yang mencapai 948. Batas berat adalah batasan yang menentukan (31 dari 33 state akhir mengangkut tepat 72).
3. Sideways-move memberi rata-rata tertinggi dan paling konsisten (938,7), disusul random-restart (930,7) dan simulated annealing (930,3), lalu steepest-ascent (926,0) dan stochastic (908,0).
4. Genetic algorithm (885,7 sampai 921,3) berada di bawah hill-climbing terbaik. Menambah generasi membantu dengan hasil yang makin kecil; menambah populasi tidak terbukti berpengaruh.
5. Steepest-ascent paling cepat (0,22 detik) dan simulated annealing paling lama (6,81 detik).

Saran:

1. Menambah jumlah run (misalnya 30 per konfigurasi) agar perbedaan antar algoritma dapat diuji secara statistik.
2. Mencoba max_sideways yang lebih besar, dan memakai sideways-move sebagai algoritma dalam random-restart.
3. Pada simulated annealing, mengembalikan state terbaik yang pernah dilihat atau menambah restart.
4. Pada GA, memakai crossover yang menjaga tata letak spasial, atau menambah move yang menukar satu kendaraan dengan beberapa kendaraan lain pada hill-climbing.
5. Menguji objective alternatif dan, bila waktu memungkinkan, multi-kapal.

BAB 6
PEMBAGIAN TUGAS

Nama	NIM	Tugas
Agung Ahmad Rifai	18224042	• Mengerjakan algoritma hill-climbing beserta bonus 2 • Mengerjakan algoritma simulated-annealing • Menuliskan laporan untuk seluruh Hill-climbing
Muhammad Zulfa Fauzan N	18224064	• Membuat dan mengelola repositori • Melakukan verifikasi pada setiap algoritma untuk memastikan kesesuaian spesifikasi dan ketahanan dalam meng-*handle edge cases* • Mengerjakan tampilan antarmuka
Bram Sebastian Pangaribuan	18224076	• Mengerjakan bagian genetic algorithm • Membuat laporan terkait genetic algorithm dan simulated annealing • Memetakan kesesuaian laporan terhadap spesifikasi • Membuat instance default

BAB 7
REFERENSI

S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ, USA: Pearson, 2020, ch. 4, sec. 4.1.

M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=oGTa9kGE3P8

M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM

M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8

M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=AdhG7HnhYrk

M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=al41TGo9J9c

W. Ben-Ameur, "Computing the initial temperature of simulated annealing," Comput. Optim. Appl., vol. 29, no. 3, pp. 369-385, Dec. 2004, doi: 10.1023/B:COAP.0000044187.23143.BD.

A. E. Eiben and J. E. Smith, Introduction to Evolutionary Computing, 2nd ed. Berlin, Germany: Springer, 2015, sec. 4.2.1 and 5.3.2.

BAB 8
LAPORAN FORM PENGGUNAAN GENERATIVE AI

Dengan ini, kami :

Nama/NIM	: Agung Ahmad Rifai 18224042 Muhammad Zulfa Fauzan N 18224064 Bram Sebastian Pangaribuan 18224076
Fakultas/Sekolah	: Sekolah Teknik Elektro dan Informatika
Kode Mata Kuliah	: IF3070
Nama Mata Kuliah	: Dasar Artifisial Intelijen
Judul Tugas/Proposal	: *Local Search* untuk Penempatan Kendaraan pada Geladak Kapal Kargo

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan : menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya	*Low* Digunakan untuk mengecek beberapa istilah asing dan ejaan yang benar dari suatu kata.
Pembuatan Teks : menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	*Moderate* Digunakan untuk melakukan parafrase agar kalimat/paragraf dalam teks lebih terstruktur dan runut.

Bantuan Diskusi dalam Penyusunan Konten : menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	*Low* Format dokumen sudah disediakan. Penggunaan lebih ke susunan konten yang ingin ditulis.
Pembuatan Gambar, Video, dan/atau Grafik : menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-
Lainnya, Jelaskan :		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026



Agung Ahmad Rifai
18224042



Muhammad Zulfa Fauzan N
18224064



Bram Sebastian Pangaribuan
18224076